

LABORATORIO: DEL SECTOR PROFESIONAL AL CÓDIGO

Autores:
Deivid Farid Ardila Herrera
Angel David Beltran Garcia
Oscar Ivan Ulises Gutierrez Palacios
Juan Camilo Vergara Tao

Docente:
Sergio Enrique Vargas Pedraza

UNIVERSIDAD NACIONAL DE COLOMBIA
FACULTAD DE INGENIERÍA
DPTO. DE INGENIERÍA DE SISTEMAS E INDUSTRIAL
Ingeniería de Software II

BOGOTÁ D.C
2026-2

Fase 1 – Dominio y problema

1.1 Definición del problema

Sector profesional
Transporte terrestre de carga pesada, logística de distribución crítica y seguridad vial vehicular mediante tecnologías de borde (Edge AI / IoT).

Problema que resuelve
En las redes viales y corredores logísticos interurbanos, la somnolencia prolongada y los microsueños representan una de las principales fuentes de siniestralidad mortal (según reportes de la ANSV, asociados a cerca de un 28% de los siniestros fatales en transporte de carga).
Los sistemas convencionales de telemática vehicular registran eventos de forma reactiva y pasiva (como frenados bruscos o excesos de velocidad ya consumados), omitiendo la degradación neurofisiológica y cognitiva previa del conductor. Además, los túneles y las zonas montañosas sin cobertura de red móvil impiden que soluciones basadas enteramente en la nube procesen el vídeo a tiempo.

El sistema resuelve esta problemática ejecutando algoritmos de visión artificial directamente en la cabina (Edge Computing) para detectar cierres oculares prolongados (Eye Aspect Ratio) o cabeceos de inmediato, emitiendo alarmas sonoras y visuales locales de forma proactiva, a la vez que sincroniza la telemetría y el escalamiento de incidentes con la central logística cuando existe conectividad disponible.

1.2 Actores del Sistema

Actores humanos

1. Conductor: conduce el tractocamión o vehículo de carga. Interactúa con la aplicación móvil/dispositivo montado en el parabrisas, calibra su postura facial inicial, recibe alertas hápticas, sonoras y visuales de emergencia, y reporta paradas de descanso o confirmaciones de alerta activa.
2. Copiloto: miembro de apoyo a bordo del vehículo; recibe notificaciones secundarias en cabina para verificar el estado de alerta del conductor si la alarma de primer nivel no es silenciada a tiempo.
3. Despachador / Monitor de flota (B2B): supervisor logístico en el centro de control que vigila en tiempo real a través del dashboard web el mapa interactivo de la flota, niveles de riesgo acumulado, alertas críticas escaladas y estado de fatiga de la tripulación activa.
4. Administrador de seguridad: configura parámetros de umbrales biométricos, turnos de descanso reglamentarios, políticas de auditoría y gestiona credenciales de acceso institucional.

Sistemas externos

1. Pipeline de visión artificial: Módulo local de inferencia facial que captura el flujo de video a 30 FPS, calcula cabeceo y genera eventos biométricos sin depender de conexión a internet.
2. Servicio de autenticación centralizada: proveedor de autenticación delegada, encargado de la gestión segura de identidades, tokens JWT, sesiones y diferenciación entre perfiles operativos y corporativos.
3. Pasarela de notificaciones push y alertas críticas: servicio encargado de enviar mensajes push de alta prioridad y notificaciones audibles remotas a copilotos y personal de contingencia ante microsueños no solventados.
4. Motor GIS y cartografía: proveedor cartográfico para renderizar ubicaciones geográficas de la flota, tramos de concentración de incidentes y cálculo de rutas seguras con puntos de descanso habilitados.

1.3 Requisitos Funcionales (RF)

- *RF01 - Autenticación y autorización de roles:* el sistema debe autenticar y autorizar a los usuarios (Conductores y Despachadores) gestionando sesiones mediante tokens JWT para restringir el acceso a los datos de telemetría y configuración.
- *RF02 - Calibración facial biométrica:* el sistema debe ejecutar una rutina de calibración previa al viaje para establecer la línea base de apertura ocular (Eye Aspect Ratio - EAR) y postura normal de cabeza del conductor bajo diferentes condiciones lumínicas.
  RF03 - Inferencia local de fatiga y microsueño (Edge AI): el sistema debe procesar en el dispositivo cliente el flujo de la cámara en tiempo real (mínimo a 30 FPS) para detectar eventos de microsueño (EAR bajo el umbral durante más de 1.5 a 2.0 segundos) y cabeceos anómalos sin requerir conectividad de red.

- *RF04 - Disparo de alarmas proactivas en cabina:* ante la detección de un microsueño o cabeceo crítico, el sistema debe disparar de forma inmediata una alarma visual con pantalla roja de alto contraste y una señal sonora estridente en el dispositivo de cabina.
- *RF05 - Confirmación y silenciamiento de emergencia:* el sistema debe permitir al conductor interactuar con la pantalla mediante un botón táctil de gran tamaño para confirmar que ha recuperado el estado de alerta; de no recibir interacción en 5 segundos, la alarma debe intensificarse.
- *RF06 - Control del ciclo de vida del viaje:* el sistema debe gestionar las transiciones válidas del ciclo de vida del monitoreo: *Sesión creada → Calibrando → Monitoreo activo → Advertencia de fatiga → Microsueño crítico → Sesión finalizada.*
- *RF07 - Transmisión y buffering offline de telemetría:* el sistema debe almacenar localmente las coordenadas GPS, métricas de EAR e incidentes cuando se transite por zonas sin señal, sincronizándolos automáticamente vía WebSockets/REST al restablecer la conexión.
- *RF08 - Escalamiento de eventos críticos a central logística:* si un evento de microsueño no es silenciado o se repite dentro de una ventana de tiempo configurable, el sistema debe despachar una alerta prioritaria vía FCM y WebSockets hacia el panel web del despachador y el teléfono del copiloto, o vía SMS a los contactos de emergencia si es un conductor particular.
- *RF09 - Visualización y monitoreo en tiempo real:* el sistema web debe presentar sobre un mapa cartográfico el listado de vehículos activos, su estado de alerta actual (Normal, Precaución, Crítico) y el historial de incidencias del turno.

1.4 Atributos de Calidad (AC)

AC01 - Rendimiento

- *Justificación:* un microsueño a 80 km/h recorre más de 22 metros por segundo; la detección y activación de la alarma en cabina no puede depender de una llamada HTTP a un servidor remoto.
- *Métrica*: la inferencia de la malla facial (Face Mesh) y la evaluación de la regla de alerta deben completarse en un tiempo $\leq 50ms$ por cuadro (frame), manteniendo una tasa mínima de 30 FPS en el dispositivo de cabina.

AC02 - Disponibilidad

- *Justificación*: las rutas de transporte pesado atraviesan sectores montañosos, desiertos y túneles con nula cobertura de telecomunicaciones, donde el riesgo de fatiga suele ser mayor.
- *Métrica*: el subsistema de detección de fatiga y activación de alarmas sonoras debe operar con 100% de autonomía sin acceso a red, persistiendo eventos en cola local para su descarga diferida.

AC03 - Seguridad biométrica

- *Justificación*: la captura de video y rasgos faciales de los operadores está sujeta a normativas de protección de datos personales y biométricos.
- *Métrica*: ninguna transmisión de video crudo ni imágenes faciales debe ser enviada a la nube; únicamente se transmitirán métricas vectoriales discretas (índices numéricos de EAR, ángulos de cabeceo y marcas de tiempo).

AC04 - Mantenibilidad del motor de inferencia

- *Justificación*: el equipo debe poder actualizar o intercambiar las bibliotecas de visión computacional sin reescribir la lógica de negocio.
- *Métrica*: el acoplamiento con la biblioteca de inferencia debe realizarse mediante interfaces y adaptadores desacoplados, requiriendo 0 modificaciones en el núcleo de dominio de alertas ante un cambio de motor.

AC05 - Usabilidad y ergonomía en cabina

- *Justificación*: la interfaz no debe encandilar la vista del conductor durante la noche ni exigir precisión táctil milimétrica en un vehículo sujeto a vibraciones constantes.
- *Métrica*: el 80% de la interfaz activa debe mantener un contraste en modo oscuro con tipografías legibles a más de 1 metro de distancia, requiriendo un único toque en un área objetivo $\geq 64 \times 64dp$ para interactuar.

AC06 - Auditabilidad y trazabilidad de incidentes

- *Justificación*: en caso de auditorías laborales o investigaciones de incidentes viales de la empresa de transporte, se requiere verificar segundo a segundo la cronología de advertencias previas.
- *Métrica*: cada incidente de fatiga debe consolidar un registro inmutable con marca de tiempo UTC exacta, ubicación GPS georreferenciada y secuencia de avisos de escalamiento para su almacenamiento en base de datos relacional.

Fase 2 – Arquitectura macro

2.1 Justificación

Para el sistema de detección de fatiga Tire, el equipo seleccionó una arquitectura de Microservicios orientada a eventos, teniendo en cuenta los atributos de calidad definidos anteriormente:

- *Rendimiento (AC01)*: la naturaleza del tráfico es radicalmente asimétrica, ya que la telemetría de cientos de camiones enviando datos de GPS, EAR y eventos cada pocos segundos es tráfico masivo y concurrente. En cambio, el CRUD de conductores y empresas es de muy bajo volumen.

  Los microservicios permiten tener un servicio de telemetría ultra liviano y optimizado, que escala horizontalmente a 10 o 20 instancias en horas de alta circulación en carretera, mientras el servicio de autenticación y configuración se queda en 1 o 2 instancias.

- *Disponibilidad (AC02)*: en un monolito, si el módulo de reportes históricos de flota o el panel administrativo sufren una consulta SQL pesada o un desbordamiento de memoria, todo el proceso del servidor se cae, interrumpiendo el flujo de recepción de alertas de emergencia y la comunicación en tiempo real con los vehículos.

  Por lo tanto, la solución más viable es desacoplar el sistema en servicios independientes, para que el servicio de alertas críticas siga corriendo de forma aislada, garantizando que ninguna alerta de microsueño se pierda.

- *Mantenibilidad (AC04)*: las flotas operan 24/7, por lo que no se puede bajar el servidor en cierto momento para actualizar la integración del proveedor cartográfico o cambiar una regla en el panel web. En el caso de microservicios, cada equipo actualiza y despliega su servicio sin interrumpir la recepción de eventos de cabina.

2.2 Componentes del sistema

Capa de Presentación

- `tire_mobile_app`*:* Aplicación móvil principal instalada en el smartphone del conductor. Actúa como el único dispositivo de captura en cabina para todos los usuarios (particulares B2C y flotas B2B). Responsabilidades:
  - Capturar video en tiempo real a través de la cámara frontal.
  - Recolectar datos de telemetría del propio teléfono (GPS, acelerómetro).
  - Emitir alarmas sonoras y vibraciones de emergencia localmente.
- `tire_edge_ai_vision`*:* Módulo de visión computacional incrustado dentro de la `tire_mobile_app`, ejecuta modelos ligeros de IA en el dispositivo para calcular el EAR (Eye Aspect Ratio) localmente, evitando latencias de red.
- `tire_web_dashboard`*:* Portal administrativo web, permite a los gestores de flota B2B visualizar el mapa interactivo, auditar rutas y recibir alertas críticas en tiempo real.

Capa de Comunicación

- `tire_api_gateway`*:* Recibe el tráfico de la app móvil y del dashboard web, enruta las peticiones de login al servicio de autenticación, encola la telemetría en `tire_broker` y gestiona conexiones persistentes (WebSockets).

Servicios Externos- Notificaciones

- `tire_fcm_notifications`*:* Servicio push para notificar a copilotos.
- `tire_sms_gateway`*:* Proveedor para enviar SMS de emergencia.

Capa de Lógica

- *`tire_auth`*: Microservicio exclusivo de Identidad y Accesos (IAM), maneja el registro, el login y la generación de token, el Gateway le consulta para validar accesos.
- *`tire_user_service`*: Gestiona el perfil B2C, preferencias e historial personal del usuario regular, además de sus contactos de emergencia.
- *`tire_fleet_service`*: Gestiona la lógica B2B, provee la información de empresas, vehículos, conductores y configuraciones de flotas.
- `tire_broker`: Buffer de mensajería (ej. Kafka, RabbitMQ) dedicado exclusivamente a amortiguar la ingesta masiva de telemetría proveniente del Gateway.
- *`tire_telemetry_service`*: procesa los eventos del broker relacionados con GPS y velocidad asíncronamente y los guarda.
- *`tire_alert_service`*: Recibe las alertas de fatiga síncronas desde el Gateway. Cruza la alerta con los perfiles (`tire_user_service` o `tire_fleet_service`) para armar el contexto del evento. Con ese contexto ejecuta el árbol de decisiones para escalar la alerta (copiloto por FCM, contactos de emergencia por SMS o despachador) y registra el incidente en `tire_alert_db`. La alarma inicial en cabina la emite la app localmente (sin red); este servicio solo decide el escalamiento.

Capa de Datos

- *`db_auth`*: Base de datos asociada al servicio de auth (dependiendo si usamos Clerk estaria asociada a este servicio).
- *`tire_user_db`*: Base de datos dedicada a los perfiles de usuarios B2C.
- *`tire_fleet_db`*: Base de datos relacional para las entidades de negocio (flotas B2B).
- *`tire_telemetry_db`*: Base de datos optimizada (Time-Series) para el alto volumen de registros generados por `tire_telemetry_service`.
- *`tire_alert_db`*: Base de datos relacional de `tire_alert_service` para registrar cada incidente con su hora, ubicación y secuencia de escalamiento (AC06).

2.3 Diagrama de Arquitectura Macro (C4 - Nivel de Contenedores)

Arquitectura

El diagrama agrupa los componentes por capas: presentación (app móvil y dashboard web), comunicación (API Gateway), lógica (microservicios) y datos (una base por servicio). La app envía cada alerta de fatiga al gateway, que la entrega a `tire_alert_service`. Este servicio consulta el perfil del conductor en `tire_user_service` o `tire_fleet_service`, decide a quién escalar (copiloto por FCM, contactos de emergencia por SMS o despachador a través del gateway) y registra el incidente en `tire_alert_db`. La telemetría sigue un camino aparte y asíncrono: pasa por `tire_broker` hasta `tire_telemetry_service`.

El diagrama de arquitectura macro de Tire (C4, nivel de contenedores) se encuentra disponible en el siguiente enlace:
https://github.com/ML-Ingesoft-II/lab3-tire-alert-service/blob/main/docs/diagramas/fase2-arquitectura-macro.svg.

Fase 3 – Componente y sub-arquitectura

3.1 Componente seleccionado

Se eligió `tire_alert_service`, el servicio que decide a quién avisar cuando un conductor tiene un microsueño y no responde a la alarma de la cabina. Esta elección se debe a que allí se encuentra la lógica fundamental de Tire. Los demás servicios definidos en la Fase 2 se encargan principalmente de guardar y consultar datos (usuarios, flotas, autenticación y telemetría), mientras que este tiene que tomar una decisión distinta según cada caso.

Esta decisión depende del perfil del conductor: si es un conductor particular que viaja solo, se notifica por SMS a sus contactos de emergencia; si viaja con copiloto, se envía una notificación push al copiloto, y si pertenece a una flota, la alerta se muestra en el dashboard del despachador (RF08). Además, cada incidente debe quedar registrado con su hora, ubicación y secuencia de avisos para permitir auditorías posteriores (AC06).

Por otra parte, es el servicio con mayor probabilidad de cambio. La llegada de un nuevo tipo de usuario, como el transporte público, o de un nuevo canal de notificación afectaría directamente este componente, por lo que su diseño debe permitir incorporar esos cambios sin modificar lo que ya funciona.

3.2 Sub-arquitectura: Hexagonal

Para la estructura interna del servicio se eligió la arquitectura hexagonal (puertos y adaptadores). Las reglas de escalamiento se ubican en un núcleo independiente de las tecnologías externas, que se comunica con el exterior únicamente mediante interfaces (puertos) implementadas por adaptadores. De esta forma, cambiar un proveedor como FCM, el servicio de SMS o la base de datos solo implica escribir un adaptador nuevo, sin modificar el núcleo, criterio que coincide con el AC04.

Se descartó la arquitectura por capas porque en ella la lógica de escalamiento dependería de la capa de datos y de los clientes de FCM y SMS, de modo que un cambio de proveedor obligaría a modificar capas de las que depende el negocio. Tampoco se eligió un estilo orientado a eventos dentro del servicio, ya que el flujo es una secuencia corta (recibir la alerta, decidir, notificar y registrar) y dividirlo en mensajes internos lo haría más difícil de seguir y de probar, sin aportar beneficios a esta escala.

Finalmente, esta estructura facilita el trabajo en la Ruta A: como el laboratorio no exige una base de datos real ni servicios externos, cada puerto de salida puede tener un adaptador simulado, lo que permite probar las reglas del núcleo sin depender de infraestructura.

3.3 Distribución de clases en la arquitectura

La siguiente tabla resume las clases e interfaces del servicio y la parte de la arquitectura a la que pertenece cada una.

Parte	Clase o interfaz	Responsabilidad
Núcleo (dominio)	`Incidente`	Representa una alerta con su hora, ubicación e historial de avisos
Núcleo (dominio)	`NivelRiesgo`	Nivel del incidente: normal, precaución o crítico (RF09)
Núcleo (dominio)	`ContextoConductor`	Perfil del conductor: particular o de flota, copiloto y contactos de emergencia
Núcleo (dominio)	`AccionEscalamiento`	Un aviso concreto: a quién se envía, por qué canal y con qué mensaje
Núcleo (dominio)	`ReglasEscalamiento`	Decide cuándo escalar (alarma no silenciada o repetida dentro de la ventana) y a quién avisar
Núcleo (aplicación)	`ServicioEscalamiento`	Caso de uso: registra el incidente, consulta el historial, decide y envía los avisos
Puerto de entrada	`ProcesarAlerta`	Única operación que el exterior puede solicitar al servicio
Puerto de salida	`ConsultorPerfil`	Obtener el perfil del conductor
Puerto de salida	`NotificadorPush`	Notificar al copiloto
Puerto de salida	`NotificadorSms`	Notificar a los contactos de emergencia
Puerto de salida	`NotificadorDashboard`	Notificar al despachador
Puerto de salida	`RepositorioIncidentes`	Guardar y consultar incidentes
Adaptador de entrada	`ControladorAlertasRest`	Recibe la alerta que llega desde `tire_api_gateway`
Adaptadores de salida	`ConsultorPerfilSimulado`, `NotificadorPushFcmSimulado`, `NotificadorSmsSimulado`,	Implementan cada puerto de salida sin infraestructura real. En producción se reemplazarían por

	`NotificadorDashboardSimulado`, `RepositorioIncidentesEnMemoria`	`tire_user_service` y `tire_fleet_service`, FCM, el proveedor de SMS, el API Gateway y `tire_alert_db`
Configuración	`ConfiguracionAlertService`	Conecta cada puerto con su adaptador y define la ventana y el número de repeticiones

3.4 Diagrama de la sub-arquitectura

Sub-arquitectura tire_alert_service

El diagrama organiza el servicio por franjas: adaptador de entrada, núcleo, adaptadores de salida y sistemas externos. La alerta llega desde `tire_api_gateway` a `ControladorAlertasRest`, que la entrega al núcleo a través del puerto `ProcesarAlerta`. `ServicioEscalamiento` aplica las reglas del dominio y utiliza los puertos de salida (`ConsultorPerfil`, `NotificadorPush`, `NotificadorSms`, `NotificadorDashboard` y `RepositorioIncidentes`), cada uno implementado por un adaptador simulado. Las líneas punteadas indican el sistema real que reemplazaría a cada adaptador: `tire_user_service` y `tire_fleet_service`, FCM, el proveedor de SMS, el API Gateway y `tire_alert_db`. Todas las dependencias apuntan hacia el núcleo, por lo que cambiar un proveedor solo requiere un adaptador nuevo.

El diagrama de la sub-arquitectura hexagonal de `tire_alert_service` se encuentra disponible en el siguiente enlace:
https://github.com/ML-Ingesoft-II/lab3-tire-alert-service/blob/main/docs/diagramas/fase3-sub-arquitectura.svg.

Fase 4 – SOLID

4.1 Diagrama de clases

El diagrama desarrolla las clases de la sección 3.3 dentro de los paquetes de la sub-arquitectura hexagonal: adaptador de entrada, aplicación (puerto de entrada y caso de uso), dominio, puertos de salida y adaptadores de salida. `ServicioEscalamiento` depende únicamente de interfaces, y las implementaciones simuladas se conectan en `ConfiguracionAlertService`.

El diagrama de clases base de `tire_alert_service` se encuentra disponible en el siguiente enlace: https://github.com/ML-Ingesoft-II/lab3-tire-alert-service/blob/main/docs/diagramas/fase4-clases-base.svg.

4.2 Evidencia de los principios SOLID

Cada principio se cumple en al menos una clase o interfaz concreta del diseño. Los principios de segregación de interfaces e inversión de dependencias incluyen además el diseño previo que los violaba (sección 4.3).

Principio	Dónde se cumple	Evidencia
S: Responsabilidad única	`ServicioEscalamiento`, `ReglasEscalamiento`, adaptadores de salida	Cada clase tiene una sola razón para cambiar: `ReglasEscalamiento` contiene las reglas del negocio (cuándo escalar y a quién avisar), `ServicioEscalamiento` coordina el flujo y cada adaptador se encarga de una sola tecnología externa. Cambiar el proveedor de SMS no afecta las reglas, y cambiar una regla no afecta las integraciones.
O: Abierto/cerrado	Puertos de salida y sus adaptadores	Un proveedor nuevo, como otro servicio de SMS o una base de datos real, se incorpora con un adaptador nuevo que implementa el puerto existente, sin modificar `ServicioEscalamiento` ni `ReglasEscalamiento`.

L: Sustitución de Liskov	`RepositorioIncidentes` y sus implementaciones	`RepositorioIncidentesEnMemoria` y una futura implementación sobre `tire_alert_db` son intercambiables porque cumplen el mismo contrato; por ejemplo, `contarRecientes` devuelve 0 cuando no hay incidentes y nunca un valor nulo. El servicio funciona igual con cualquiera de las dos.
I: Segregación de interfaces	`ConsultorPerfil`, `RepositorioIncidentes`, `NotificadorPush`, `NotificadorSms`, `NotificadorDashboard`	Los puertos de salida son pequeños y específicos, de modo que ningún adaptador implementa métodos que no usa: el repositorio solo guarda y cuenta incidentes, y cada notificador solo envía por su canal.
D: Inversión de dependencias	`ServicioEscalamiento`, `ConfiguracionAlertService`	El servicio depende únicamente de interfaces definidas por el núcleo. Las implementaciones concretas se crean en `ConfiguracionAlertService` y se le entregan al servicio desde afuera.

4.3 Antes y después

Los siguientes fragmentos en pseudocódigo muestran dos versiones previas del diseño que violaban un principio y cómo se corrigieron.

Caso 1: segregación de interfaces en los puertos de salida
Antes: Un único puerto agrupaba todo lo que el núcleo necesitaba del exterior. Cada adaptador estaba obligado a implementar métodos que no le correspondían.

```
TypeScript
interfaz InfraestructuraAlertas
  obtenerContexto(conductorId), guardar(incidente), contarRecientes(conductorId,
desde)
  enviarPush(accion), enviarSms(accion), enviarDashboard(accion)

clase NotificadorSmsSimulado implementa InfraestructuraAlertas
  enviarSms(accion)                       -> envía el mensaje
  obtenerContexto(...), guardar(...), ... -> error: no aplica
```

Después: El puerto se dividió en interfaces pequeñas, y cada adaptador implementa solo la que le corresponde.

```
TypeScript
interfaz NotificadorSms: enviar(accion)
interfaz RepositorioIncidentes: guardar(incidente), contarRecientes(conductorId,
desde)

clase NotificadorSmsSimulado implementa NotificadorSms
  enviar(accion) -> envía el mensaje
```

Caso 2: inversión de dependencias en los servicios externos
Antes: El servicio creaba por sí mismo los clientes concretos de FCM y SMS y la conexión a la base de datos. No era posible probarlo sin esos sistemas ni cambiar de proveedor sin editar la clase.

```
TypeScript
clase ServicioEscalamiento
  fcm <- nuevo ClienteFirebase(claveApi)
```

```
  sms <- nuevo ClienteTwilio(cuenta, token)
  baseDatos <- nueva ConexionPostgres("tire_alert_db")
```

Después: El servicio recibe interfaces en su constructor, y `ConfiguracionAlertService` decide qué implementación entregar: los adaptadores simulados en el laboratorio o los reales en producción.

TypeScript

```
clase ServicioEscalamiento
  constructor(perfiles: ConsultorPerfil, repositorio: RepositorioIncidentes,
              push: NotificadorPush, sms: NotificadorSms,
              dashboard: NotificadorDashboard)

clase ConfiguracionAlertService
  crearServicio() -> nuevo ServicioEscalamiento(
    nuevo ConsultorPerfilSimulado(), nuevo RepositorioIncidentesEnMemoria(),
    nuevo NotificadorPushFcmSimulado(), nuevo NotificadorSmsSimulado(),
    nuevo NotificadorDashboardSimulado())
```

Fase 5 – Patrones de diseño

Se aplican seis patrones, dos por familia: Factory Method y Builder (creacionales), Adapter y Decorator (estructurales), y Strategy y State (de comportamiento). Cada ficha indica el problema del diseño base de la fase 4 que resuelve el patrón, y su diagrama muestra el rol de cada clase con un estereotipo. Al final se presenta el diagrama completo.

5.1 Strategy

Elemento	Descripción
Familia	Comportamiento
Problema	En el diseño base, `ReglasEscalamiento` decide a quién avisar con una condición por cada tipo de perfil (RF08). Cada perfil nuevo, como el transporte público, obliga a modificar ese método y a volver a probar los casos que ya funcionaban.
Solución	Cada forma de escalar es una clase que implementa `PoliticaEscalamiento`. El servicio pide las acciones a la política del conductor sin saber de qué perfil se trata.
Alternativa descartada	Template Method: las políticas no comparten pasos comunes que se puedan heredar de una clase base, porque cada una decide sus avisos de forma distinta. Con Strategy, además, cada política se prueba por separado.
Participantes	Context: `ServicioEscalamiento`. Strategy: `PoliticaEscalamiento`. ConcreteStrategy: `PoliticaParticularSolo`, `PoliticaParticularConCopiloto` y `PoliticaFlota`.
Relación con SOLID	Abierto/cerrado: un perfil nuevo se agrega con una clase nueva, sin modificar el servicio. Este caso se usará en la prueba unitaria de abierto/cerrado de la fase 6.

El diagrama del patrón Strategy se encuentra disponible en el siguiente enlace: https://github.com/ML-Ingesoft-II/lab3-tire-alert-service/blob/main/docs/diagramas/fase5-strategy.svg.

5.2 Factory Method

Elemento	Descripción
Familia	Creacional
Problema	Al quitar las condiciones por perfil de `ReglasEscalamiento`, alguien debe decidir qué política usar para cada conductor. Si lo hace `ServicioEscalamiento`, la condición solo

	cambia de lugar y el servicio vuelve a depender de todas las políticas concretas.
Solución	`FabricaPoliticas` concentra la creación en el método `crearPara`(contexto), que revisa el perfil del conductor y devuelve la política como `PoliticaEscalamiento`. El servicio solo conoce la interfaz.
Alternativa descartada	Abstract Factory: no hay familias de objetos relacionados que deban crearse juntos; por cada incidente se necesita una sola política.
Participantes	Client: `ServicioEscalamiento`. Creator: `FabricaPoliticas`, cuyo método `crearPara` puede sobrescribirse. ConcreteCreator: una subclase de `FabricaPoliticas` que sobrescribe `crearPara` para un perfil nuevo. Product: `PoliticaEscalamiento`. ConcreteProduct: las tres políticas de la sección 5.1.
Relación con SOLID	Inversión de dependencias, porque el servicio depende de la interfaz y no de las clases concretas, y responsabilidad única, porque la elección de la política queda en un solo lugar.

El diagrama del patrón Factory Method se encuentra disponible en el siguiente enlace: https://github.com/ML-Ingesoft-II/lab3-tire-alert-service/blob/main/docs/diagramas/fase5-factory-method.svg.

5.3 State

Elemento	Descripción
Familia	Comportamiento
Problema	Un incidente pasa por varias etapas: se detecta, se escala cuando las reglas lo deciden y se marca como atendido cuando alguien responde. Su nivel de riesgo y las operaciones que acepta dependen de la etapa. Con un campo de estado y condiciones en cada método, `Incidente` repetiría las mismas comparaciones y permitiría transiciones inválidas, como volver a escalar un incidente ya atendido.
Solución	Cada etapa es una clase que implementa `EstadoIncidente` y decide qué hacer y a qué estado pasar: un incidente detectado (precaución) pasa a escalado (crítico) al registrar sus acciones, y uno atendido (normal) ignora nuevos escalamientos. `Incidente` delega en su estado actual, del que también se obtiene el nivel de riesgo (RF09); cuándo escalar lo decide `ReglasEscalamiento`.
Alternativa descartada	Un enum con un switch en cada método: un estado nuevo, como "falsa alarma", obligaría a revisar todos los switch.
Participantes	Context: `Incidente`. State: `EstadoIncidente`. ConcreteState: `EstadoDetectado`, `EstadoEscalado` y `EstadoAtendido`.
Relación con SOLID	Responsabilidad única, porque cada estado contiene solo sus reglas, y abierto/cerrado, porque un estado nuevo es una clase nueva.

El diagrama del patrón State se encuentra disponible en el siguiente enlace: https://github.com/ML-Ingesoft-II/lab3-tire-alert-service/blob/main/docs/diagramas/fase5-state.svg.

5.4 Builder

Elemento	Descripción
Familia	Creacional
Problema	Un `Incidente` se arma con varios datos de la alerta (conductor, tipo de evento, ubicación, hora y si la alarma fue silenciada), y algunos pueden faltar, como la ubicación sin señal GPS. Un constructor con cinco parámetros seguidos es fácil de usar mal, y crear el objeto vacío para completarlo después permitiría incidentes incompletos.
Solución	`IncidenteBuilder` recibe los datos paso a paso con métodos de nombre claro, y `construir()` verifica los datos obligatorios antes de crear el incidente, que inicia en estado detectado.
Alternativa descartada	Constructores sobrecargados, uno por cada combinación de datos opcionales: su número crece con cada campo nuevo.
Participantes	Client: `ServicioEscalamiento`. Builder: `IncidenteBuilder`. Product: `Incidente`. No se usa un director, porque hay una sola forma de armar el incidente.

Relación con SOLID	Responsabilidad única: la validación de los datos de entrada sale de `Incidente`, que se encarga solo de su comportamiento.

El diagrama del patrón Builder se encuentra disponible en el siguiente enlace: https://github.com/ML-Ingesoft-II/lab3-tire-alert-service/blob/main/docs/diagramas/fase5-builder.svg.

5.5 Adapter

Elemento	Descripción
Familia	Estructural
Problema	Los servicios externos de notificación tienen su propia interfaz: el cliente de FCM envía con `send(token, payload)` y el proveedor de SMS con `sendMessage(numero, texto)`, mientras que el núcleo espera `enviar(accion)`. En el laboratorio estos clientes se reemplazan por versiones falsas, como permite el enunciado.
Solución	Cada adaptador implementa `Notificador` y traduce la acción de escalamiento al formato del cliente externo: token del dispositivo, número de teléfono y texto del mensaje.
Alternativa descartada	Facade: no se trata de simplificar un subsistema complejo, sino de convertir una interfaz ajena a la que el núcleo ya definió.
Participantes	Client: `ServicioEscalamiento`. Target: `Notificador`. Adapter: `NotificadorPushFcmSimulado` y `NotificadorSmsSimulado`. Adaptee: `ClienteFcmFalso` y `ProveedorSmsFalso`.
Relación con SOLID	Inversión de dependencias: el núcleo define la interfaz y los detalles de cada proveedor quedan en el adaptador. Cambiar de proveedor solo requiere un adaptador nuevo.

El diagrama del patrón Adapter se encuentra disponible en el siguiente enlace: https://github.com/ML-Ingesoft-II/lab3-tire-alert-service/blob/main/docs/diagramas/fase5-adapter.svg.

5.6 Decorator

Elemento	Descripción
Familia	Estructural
Problema	Los envíos por push o SMS pueden fallar por causas temporales, como la falta de red, y una alerta de microsueño no puede perderse. Si cada adaptador implementa sus propios reintentos, la misma lógica se repite en todos y se mezcla con la traducción al proveedor.
Solución	`NotificadorConReintento` implementa `Notificador` y envuelve a otro notificador: si el envío falla, lo repite hasta un número fijo de intentos. El servicio no nota la diferencia, y `ConfiguracionAlertService` decide a qué canales se aplica. Para esto, los tres puertos de notificación de la fase 4, que ya tenían el mismo método `enviar(accion)`, se agrupan bajo la interfaz común `Notificador`.
Alternativa descartada	Herencia, con una subclase con reintento por cada notificador: duplicaría clases y no permitiría combinar comportamientos, como reintentar y registrar cada envío.
Participantes	Client: `ServicioEscalamiento`. Component: `Notificador`. ConcreteComponent: `NotificadorPushFcmSimulado`, `NotificadorSmsSimulado` y `NotificadorDashboardSimulado`. Decorator: `NotificadorConReintento`.
Relación con SOLID	Abierto/cerrado, porque se agrega comportamiento sin modificar los notificadores, y responsabilidad única, porque el reintento queda en una sola clase.

El diagrama del patrón Decorator se encuentra disponible en el siguiente enlace: https://github.com/ML-Ingesoft-II/lab3-tire-alert-service/blob/main/docs/diagramas/fase5-decorator.svg.

5.7 Diagrama de clases con patrones

El diagrama reúne los seis patrones sobre el diseño base de la fase 4. Respecto a ese diseño hay dos cambios: `ReglasEscalamiento` solo decide cuándo escalar, mientras que a quién avisar pasa a las políticas, y los tres puertos de notificación comparten la interfaz `Notificador`. Para facilitar la lectura, las clases muestran solo los métodos relacionados con los patrones; los atributos completos están en el diagrama de la sección 4.1. Este diagrama es la base del código de la fase 6.

El diagrama de clases con todos los patrones se encuentra disponible en el siguiente enlace: https://github.com/ML-Ingesoft-II/lab3-tire-alert-service/blob/main/docs/diagramas/fase5-clases-patrones.svg.

Fase 6 – Evidencia Ruta A

Para validar la correcta implementación de la sub-arquitectura hexagonal, los principios SOLID y los seis patrones de diseño seleccionados, se desarrolló una suite de pruebas automatizadas con Vitest en TypeScript sobre Node.js 22. Siguiendo los lineamientos de la Ruta A, la verificación se realiza sin depender de infraestructura real (bases de datos o APIs cloud activas), utilizando un repositorio en memoria y adaptadores simulados.

El código fuente del servicio se encuentra disponible en el repositorio del proyecto: https://github.com/ML-Ingesoft-II/lab3-tire-alert-service.

La siguiente tabla relaciona cada patrón con el archivo y la clase que lo implementan en el repositorio

Patrón	Archivo y clase
Strategy	`src/dominio/politicas/PoliticaEscalamiento.ts`; `PoliticaParticularSolo`.ts, `PoliticaParticularConCopiloto`.ts y `PoliticaFlota`.ts; `src/aplicacion/ServicioEscalamiento.ts`
Factory Method	`src/aplicacion/FabricaPoliticas.ts` (`crearPara`); subclase `FabricaPoliticasExtendida` en `tests/abierto-cerrado.test.ts`
State	`src/dominio/incidente/Incidente.ts`; `src/dominio/incidente/estados/` (`EstadoIncidente`, `EstadoDetectado`, `EstadoEscalado`, `EstadoAtendido`)
Builder	`src/dominio/incidente/IncidenteBuilder.ts`; `src/dominio/incidente/Incidente.ts`
Adapter	`src/aplicacion/puertos/salida/Notificador.ts`; `src/adaptadores/salida/notificadores/NotificadorPushFcmSimulado.ts` y `NotificadorSmsSimulado`.ts; `src/adaptadores/salida/externos/ClienteFcmFalso.ts` y `ProveedorSmsFalso`.ts
Decorator	`src/aplicacion/puertos/salida/Notificador.ts`; `src/adaptadores/salida/notificadores/NotificadorConReintento.ts`

6.1 Verificación de tipos estáticos (TypeScript)

Para asegurar la coherencia arquitectónica y la ausencia de errores en los contratos de interfaz antes de la ejecución de pruebas, se utiliza el compilador de TypeScript en modo de verificación estática (`tsc --noEmit`):

```
acbel@192:~/Documents/Proyectos/Ingesoft_II/lab3-tire-alert-service$ npm run typecheck

> lab3-tire-alert-service@0.1.0 typecheck
> tsc --noEmit
```

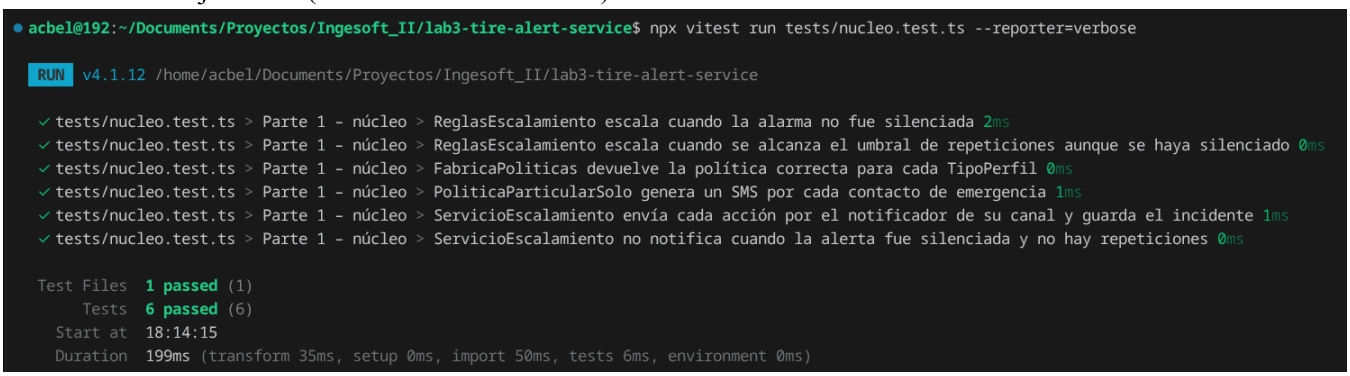
6.2 Evidencia de la Parte 1: Núcleo de Decisión (Strategy y Factory Method)

En esta parte se verificaron de forma aislada las reglas de negocio de escalamiento y el desacoplamiento de perfiles:

- *`ReglasEscalamiento`*: Se comprueba que escala de inmediato si la cabina no silenció la alarma (!silenciada), o por acumulación si se alcanza el umbral configurado (recientes + 1 >= 3).
- *`PoliticaEscalamiento` (Strategy):* Se valida que cada estrategia (`PoliticaParticularSolo`, `PoliticaParticularConCopiloto`, `PoliticaFlota`) emita las acciones correspondientes a su canal (SMS, PUSH o DASHBOARD), formateando el tipo de evento y coordenadas, y lanzando errores si faltan datos de contacto.
- *`FabricaPoliticas` (Factory Method)*: Verifica que se instancie la clase concreta adecuada para cada valor de `TipoPerfil`.

- *`ServicioEscalamiento`*: Se comprueba el flujo del caso de uso mediante dobles de prueba (*stubs* y *spies*). Se verifica que transicione el incidente a estado crítico antes de despachar las alertas y que en el bloque finally se invoque siempre repositorio.guardar(incidente) para asegurar la auditabilidad (AC06).

Resultado de la ejecución (tests/`nucleo.test.ts`):

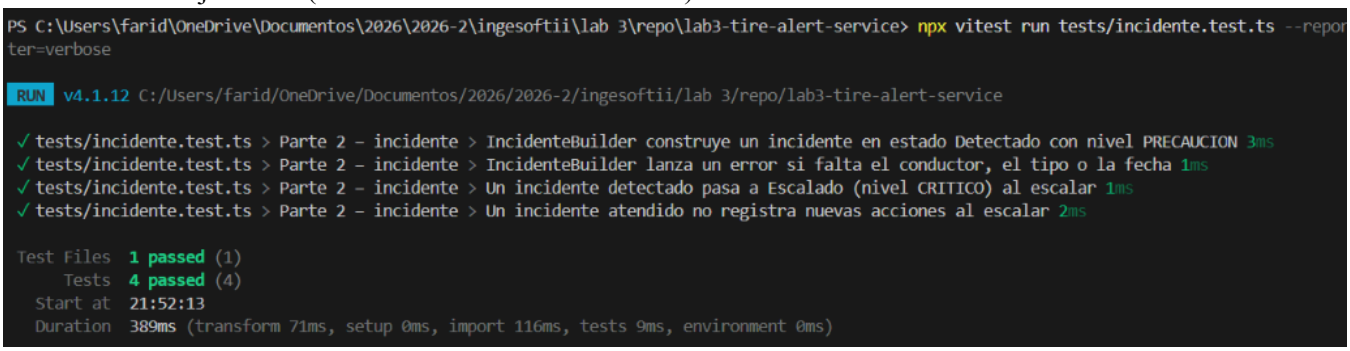


6.3 Evidencia de la Parte 2: Ciclo de Vida y Construcción del Incidente (State y Builder)

En esta sección se evalúa la consistencia del objeto `Incidente`:

- *`IncidenteBuilder` (Builder)*: Valida que la construcción paso a paso requiera estrictamente conductorId, tipo y fecha, impidiendo la creación de incidentes inconsistentes.
- *`EstadoIncidente` (State):* Comprueba el ciclo de vida Detectado (PRECAUCIÓN) → Escalado (CRÍTICO) → Atendido (NORMAL), verificando que un incidente atendido ignore nuevos intentos de escalamiento.

Resultado de la ejecución (`tests/incidente.test.ts`):



6.4 Evidencia de la Parte 3: Adaptadores de Salida y Resiliencia (Adapter y Decorator)

Se evalúa la integración tecnológica simulada:

- *`NotificadorPushFcmSimulado` y `NotificadorSmsSimulado` (Adapter):* Traducen la acción de dominio a las interfaces nativas `send(token, payload)` y `sendMessage(numero, texto)` de los SDKs externos.
- *`NotificadorConReintento` (Decorator):* Envuelve a los notificadores para realizar hasta 3 intentos en total (2 reintentos) ante fallas temporales antes de propagar la excepción.
- *`RepositorioIncidentesEnMemoria` y `ConsultorPerfilSimulado`*: Verifican el filtrado temporal por conductor dentro de la ventana de tiempo y la consulta de perfiles sin infraestructura externa.

Resultado de la ejecución (tests/`adaptadores.test.ts`):

```
PS C:\Users\jcver\UNAL\lab3-tire-alert-service> npx vitest run tests/adaptadores.test.ts --reporter=verbose

 RUN  v4.1.12 C:/Users/jcver/UNAL/lab3-tire-alert-service


 ❯ tests/adaptadores.test.ts [queued]

 ❯ tests/adaptadores.test.ts 0/8
 ✓ tests/adaptadores.test.ts > Adaptadores de salida > NotificadorSmsSimulado traduce la acción a sendMessage(numero, texto) 3ms
 ✓ tests/adaptadores.test.ts > Adaptadores de salida > NotificadorSmsSimulado lanza un error si el proveedor responde ok = false 2ms
 ✓ tests/adaptadores.test.ts > Adaptadores de salida > NotificadorPushFcmSimulado traduce la acción a send(token, { title, body }) 1ms
 ✓ tests/adaptadores.test.ts > Adaptadores de salida > NotificadorDashboardSimulado publica la acción para el despachador 0ms
 ✓ tests/adaptadores.test.ts > Adaptadores de salida > NotificadorConReintento reintenta hasta lograr el envío cuando el proveedor falla temporalmente 0ms
 ✓ tests/adaptadores.test.ts > Adaptadores de salida > NotificadorConReintento relanza el error si fallan todos los intentos 0ms
 ✓ tests/adaptadores.test.ts > Adaptadores de salida > RepositorioIncidentesEnMemoria cuenta solo los incidentes del conductor dentro de la ventana 1ms
 ✓ tests/adaptadores.test.ts > Adaptadores de salida > ConsultorPerfilSimulado devuelve el perfil registrado y lanza un error si no existe 0ms

  Test Files  1 passed (1)
       Tests  8 passed (8)
    Start at  23:13:51
    Duration  367ms (transform 94ms, setup 0ms, import 138ms, tests 9ms, environment 0ms)
```

6.5 Evidencia de la Parte 4: Adaptador de Entrada (Controlador REST)

Se valida la recepción de la alerta tal como llegaría desde `tire_api_gateway`. El controlador no levanta un servidor HTTP: recibe el JSON, lo valida y lo entrega al puerto `ProcesarAlerta`.

- *`ControladorAlertasRest`*: Verifica que el payload JSON contenga los atributos obligatorios, valida la enumeración del evento, convierte la fecha ISO a tipo Date nativo y entrega la estructura tipada a `ProcesarAlerta`.

Resultado de la ejecución (`tests/controlador.test.ts`):

```
oscar@192:~/Escritorio/ingesoft2/lab3-tire-alert-service$ npx vitest run tests/controlador.test.ts --reporter=verbose

 RUN  v4.1.12 /home/oscar/Escritorio/ingesoft2/lab3-tire-alert-service

 ✓ tests/controlador.test.ts > ControladorAlertasRest > rechaza un JSON sin conductorId 19ms
 ✓ tests/controlador.test.ts > ControladorAlertasRest > convierte el JSON en AlertaFatiga y la entrega al caso de uso 8ms

 Test Files  1 passed (1)
      Tests  2 passed (2)
   Start at  23:28:38
   Duration  588ms (transform 81ms, setup 0ms, import 125ms, tests 34ms, environment 0ms)
```

6.6 Evidencia de la Suite Completa y Demostración en Consola

A. Ejecución de la suite completa de pruebas:
Se ejecuta la suite completa de pruebas automatizadas, que cubre las cuatro partes y la prueba de evolución:

```
oscar@192:~/Escritorio/ingesoft2/lab3-tire-alert-service$ npm test

> lab3-tire-alert-service@0.1.0 test
> vitest run


 RUN  v4.1.12 /home/oscar/Escritorio/ingesoft2/lab3-tire-alert-service

 ✓ tests/controlador.test.ts (2 tests) 22ms
 ✓ tests/incidente.test.ts (4 tests) 34ms
 ✓ tests/abierto-cerrado.test.ts (1 test) 20ms
 ✓ tests/nucleo.test.ts (6 tests) 27ms
 ✓ tests/adaptadores.test.ts (8 tests) 27ms

 Test Files  5 passed (5)
      Tests  21 passed (21)
   Start at  23:29:48
   Duration  877ms (transform 561ms, setup 0ms, import 1.02s, tests 130ms, environment 3ms)
```

B. Demostración interactiva de extremo a extremo (`npm start`):
Al ejecutar el punto de entrada principal *`src/main.ts`*, el sistema procesa una alerta crítica no silenciada para cada uno de los tres perfiles del sistema:

```
Alerta de microsueño de c-solo
  [SMS] a +573001112233: Alerta de fatiga: MICROSUENO detectado en (4.65, -74.06)
  [SMS] a +573004445566: Alerta de fatiga: MICROSUENO detectado en (4.65, -74.06)
  -> escalado: true, nivel: CRITICO, avisos: 2

Alerta de microsueño de c-copiloto
  [FCM] push a token-copiloto-01: Alerta de fatiga: MICROSUENO detectado en (4.65, -74.06)
  -> escalado: true, nivel: CRITICO, avisos: 1

Alerta de microsueño de c-flota
  [DASHBOARD] a flota-transandina: Alerta de fatiga: MICROSUENO detectado para la flota flota-transandina en (4.65, -74.06)
  -> escalado: true, nivel: CRITICO, avisos: 1
```

6.7 Verificación de la Prueba de Evolución

Como evidencia práctica del principio Abierto/Cerrado (OCP), la prueba *`tests/abierto-cerrado.test.ts`* demuestra que ante la llegada de un nuevo modelo de negocio (Transporte Público, Fase 7.1 del informe), basta con crear la estrategia `PoliticaTransportePublico` y una subclase de `FabricaPoliticas` que sobrescribe `crearPara`, sin modificar `ServicioEscalamiento` ni `FabricaPoliticas`. Como la prueba no modifica src/, el nuevo valor de `TipoPerfil` se simula; en el sistema se agregaría al enum (sección 7.1).

```
✓ tests/abierto-cerrado.test.ts > Abierto/cerrado > Se agrega una nueva política (Strategy) sin modificar ServicioEscalamiento 16ms

 Test Files  1 passed (1)
      Tests  1 passed (1)
   Start at  23:35:52
   Duration  734ms (transform 157ms, setup 0ms, import 282ms, tests 23ms, environment 0ms)
```

Fase 7 – Prueba de evolución (“¿Qué pasa si. . . ?”)

7.1 Nuevo tipo de usuario: transporte público

Las empresas de transporte público quieren usar Tire. Cuando un conductor de bus tiene un microsueño, la alerta debe llegar a la central de operación y al supervisor de la ruta.

- Se crean: `PoliticaTransportePublico`, que implementa `PoliticaEscalamiento`, y una subclase de `FabricaPoliticas` que sobrescribe `crearPara` para el nuevo perfil.
- Se modifican: el enum `TipoPerfil`, que agrega el nuevo valor, y `ConfiguracionAlertService`, que entrega la nueva fábrica al servicio.
- No se tocan: `FabricaPoliticas`, `ServicioEscalamiento`, `ReglasEscalamiento`, `Incidente` y sus estados, los notificadores y los demás adaptadores.

7.2 Nuevo canal de notificación: WhatsApp

Se decide avisar a los contactos de emergencia por WhatsApp, además de SMS.

- *Se crean:* `NotificadorWhatsAppSimulado`, un adaptador que implementa `Notificador`, y `ClienteWhatsAppFalso`.
- *Se modifican*: `ConfiguracionAlertService`, que registra el nuevo canal y puede envolverlo con `NotificadorConReintento`; la política que use el canal, como `PoliticaParticularSolo`, que agrega la acción por WhatsApp, y el enum `Canal` de `src/dominio/tipos.ts`, que incorpora el valor WHATSAPP.
- *No* se *tocan*: `ServicioEscalamiento`, `NotificadorConReintento`, los demás adaptadores, `Incidente` y sus estados.

7.3 Cambio de proveedor de SMS

El proveedor de SMS simulado se reemplaza por un proveedor real para salir a producción.

- *Se crean*: `NotificadorSmsTwilio`, un adaptador que implementa `Notificador` y traduce la acción al formato del nuevo proveedor.
- *Se modifican*: `ConfiguracionAlertService`, que entrega el nuevo adaptador en lugar del simulado.
- *No* se *tocan*: `ServicioEscalamiento`, las políticas, `Incidente` y sus estados, ni las pruebas del núcleo, que siguen usando los adaptadores simulados.

En los tres escenarios el cambio se concentra en clases nuevas, en el punto de ensamblaje (`ConfiguracionAlertService`) y en ajustes puntuales de tipos o de la política que usa un canal nuevo. `ServicioEscalamiento` no se modifica en ningún caso, lo que confirma lo planteado en las fases 4 y 5.

Preguntas de reflexión

1. ¿Qué patrón les resultó más natural de aplicar en su dominio y cuál fue el más difícil de justificar? ¿Por qué? ***R/*** El patrón más natural fue Strategy, porque el requisito RF08 ya define una forma distinta de escalar según el perfil del conductor, y cada forma se convirtió directamente en una política. El más difícil de justificar fue Builder: `Incidente` no tiene una gran cantidad de campos, y su uso se sostiene sobre todo por los datos opcionales, como la ubicación, y por la validación de los datos obligatorios antes de crear el objeto.

2. ¿En qué se parecen y en qué se diferencian Strategy y State? Respondan con un ejemplo de su propio proyecto. ***R/*** Ambos delegan el comportamiento en una interfaz con varias implementaciones. La diferencia está en quién decide el cambio y cuándo. En Strategy, la implementación se elige desde afuera y no cambia durante el proceso: `FabricaPoliticas` escoge `PoliticaFlota` para un conductor de flota y esa política se usa para todo el incidente. En State, el propio objeto cambia de comportamiento a medida que ocurren cosas: un `Incidente` pasa de `EstadoDetectado` a `EstadoEscalado` y luego a `EstadoAtendido`, y cada estado decide cuál sigue.

3. El estilo orientado a eventos y el patrón Observer resuelven un problema parecido. ¿En qué escala trabaja cada uno? ***R/*** Observer trabaja dentro de un mismo proceso, entre objetos en memoria; por ejemplo, `Incidente` podría avisar a varios observadores cuando cambia de estado. El estilo orientado a eventos trabaja a escala de arquitectura, entre servicios independientes que se comunican por mensajes a través de un intermediario. En Tire, ese papel lo cumple `tire_broker` con la telemetría: la app la publica y `tire_telemetry_service` la consume sin que ninguno conozca al otro. Las alertas, en cambio, no pasan por el broker: viajan de forma síncrona por el gateway hasta `tire_alert_service`, porque no pueden esperar en una cola.

4. ¿Qué patrón de su diseño eliminarían si el sistema fuera mucho más pequeño? ¿Qué costo tiene un patrón innecesario? ***R/*** Si el sistema fuera mucho más pequeño, se eliminarían Builder y Decorator: el incidente podría crearse con un constructor simple y los reintentos no serían críticos. Factory Method también podría reemplazarse por un mapa de perfil a política. Un patrón innecesario agrega clases e indirecciones que hacen el código más difícil de leer y de mantener, sin un beneficio real a cambio.

5. Si tuvieran que migrar su arquitectura de monolito a microservicios (o al revés), ¿qué partes del diseño se mantendrían gracias a SOLID y a los patrones? ***R/*** Se mantendría el núcleo completo (`Incidente` con sus estados, las políticas de escalamiento y `ServicioEscalamiento`), porque solo depende de los puertos que él mismo define (inversión de dependencias). Al pasar de microservicios a un monolito, únicamente cambiarían los adaptadores: por ejemplo, `ConsultorPerfil` dejaría de llamar a `tire_user_service` por la red y consultaría directamente el módulo de usuarios. La segregación de interfaces y los patrones Adapter y Strategy permiten hacer ese cambio sin tocar las reglas del negocio.

Referencias

La Patria. (2025, 3 de octubre). Microsueños y fatiga: la causa oculta tras accidentes de transporte pesado en Colombia. Pulzo. https://www.pulzo.com/nacion/microsuenos-y-fatiga-la-causa-oculta-tras-accidentes-de-transporte-pesado-en-colombia-PP4825009A